TI-DE

THREAT INTELLIGENCE & DETECTION ENGINEERING

TLP:WHITE

CVE-2026-33634
LiteLLM PyPI Supply Chain Attack

TeamPCP — Multi-Stage Credential Harvesting & Backdoor Implant

REPORT DATE	ATTACK WINDOW	CVSS SCORE	PLATFORMS COVERED
2026-03-26	2026-03-24 10:39–16:00 UTC	9.4 — Critical	9 / 9

DETECTION RULES

47 rules total

● CRITICAL — ACTIVE EXPLOITATION

01 Executive Summary

CRITICAL

CVE-2026-33634 is a confirmed supply chain attack against the **LiteLLM PyPI package** (versions 1.82.7 and 1.82.8) executed by threat actor **TeamPCP**. The malicious packages were live on PyPI for approximately **5.5 hours** on March 24, 2026, during which they deployed a three-stage credential-harvesting backdoor via a Python startup hook (`.pth` file). With **95 million monthly downloads**, the blast radius is enormous. This campaign is the third in a chain: Aqua Security Trivy (March 19) → Checkmarx GitHub Actions (March 23) → LiteLLM (March 24). Any organization that installed LiteLLM during the attack window, or has unpinned transitive dependencies, **must assume full credential compromise** and initiate immediate incident response.

02 CVE Details & Threat Actor Profile

CVSS SCORE

9.4

Critical — Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L

AFFECTED PACKAGES

litellm==1.82.7
litellm==1.82.8

THREAT ACTOR

TeamPCP

Financially motivated, suspected state-nexus

AFFECTED SECTORS

AI/ML · Cloud-Native · DevOps
CI/CD · GitHub Actions users

PATCH / REMEDIATION

Upgrade to litellm ≥ 1.83.0

Verify no compromise before upgrading

COMMUNITY DETECTION

ReversingLabs · Snyk · Datadog
Kaspersky · Wiz · Sysdig · Sonatype

MITRE ATT&CK Techniques

- T1546.018 Python Startup Hooks (.pth)
- T1552.001 Credentials in Files
- T1041 C2 Exfiltration
- T1547.016 Systemd Persistence
- T1087 Account Discovery
- T1005 Data from Local System
- T1555 Credential Store Access
- T1610 Kubernetes Priv-Esc

Attack Flow — 3 Stages

STAGE 1 · COLLECTION

Auto-Execute on Python Start

- .pth hook fires without import
- SSH keys & .env files
- AWS / GCP / Azure credentials
- Kubernetes kubeconfig
- GitHub PATs & CI/CD tokens
- Crypto wallets, shell history
- Completes in ~3 seconds

→

STAGE 2 · EXFILTRATION

Encrypted C2 Transmission

- AES-256-CBC encryption
- Hardcoded RSA public key
- Double base64 to evade detection
- C2: models.litellm.cloud
- Secondary: checkmarx.zone
- IP: 45.148.10.212

→

STAGE 3 · PERSISTENCE

Backdoor & Lateral Movement

- ~/.config/sysmon/sysmon.py
- systemd user service
- C2 poll every 5 minutes
- K8s privileged pod creation
- Pattern: node-setup-*
- Namespace: kube-system

03 Indicators of Compromise

15 IOCs

TYPE	INDICATOR	DESCRIPTION	CONFIDENCE	SOURCE	FIRST SEEN
DOMAIN	`models.litellm.cloud`	TeamPCP C2 (typosquat, NOT litellm.ai)	HIGH	ReversingLabs	2026-03-24
DOMAIN	`checkmarx.zone`	TeamPCP C2 (typosquat)	HIGH	Sysdig	2026-03-24
IP	`45.148.10.212`	TeamPCP staging (scan.aquasecurity.org)	HIGH	Snyk	2026-03-19
IP	`83.142.209.11`	TeamPCP staging (checkmarx.zone)	HIGH	Sysdig	2026-03-23
SHA256	`ceNa7wMJnNHy1kRnNCcwJaFjWX3pORLfMh7xGL8TUjg`	litellm_init.pth malicious payload (34,628 bytes)	HIGH	FutureSearch	2026-03-24
FILE	`litellm_init.pth`	TeamPCP .pth hook file in site-packages	HIGH	Multiple vendors	2026-03-24
FILE	`~/.config/sysmon/sysmon.py`	TeamPCP persistent backdoor script	HIGH	ReversingLabs	2026-03-24
FILE	`~/.config/systemd/user/sy`	TeamPCP systemd persistence unit	HIGH	Datadog	2026-03-24

TYPE	INDICATOR	DESCRIPTION	CONFIDENCE	SOURCE	FIRST SEEN
	`smon.service`				
FILE	`/tmp/pglog`	TeamPCP temporary payload drop	MEDIUM	Sonatype	2026-03-24
FILE	`/tmp/.pg_state`	TeamPCP state tracker file	MEDIUM	Sonatype	2026-03-24
PACKAGE	`litellm==1.82.7`	Malicious PyPI package version	HIGH	PyPI / Checkmarx	2026-03-24
PACKAGE	`litellm==1.82.8`	Malicious PyPI package (.pth variant)	HIGH	PyPI / ReversingLabs	2026-03-24
K8S	`node-setup-* (kube-system)`	TeamPCP K8s privileged pod pattern	HIGH	ARMO	2026-03-24
ARCHIVE	`tpcp.tar.gz`	TeamPCP malware archive	HIGH	Kaspersky	2026-03-24
CVE	`CVE-2026-33634`	NVD entry — CVSS 9.4 Critical	HIGH	NVD	2026-03-24

04 Detection Rules — 9 Platforms

47 Rules

Σ Sigma — Generic SIEM

.yml 5 rules

Sigma rules provide a vendor-agnostic detection format that converts to any SIEM via pySigma/sigmac. These 5 rules cover the full TeamPCP attack chain: Python malware execution patterns (T1546.018), .pth hook file creation in site-packages, systemd backdoor persistence, Python reading credential files (T1552), and DNS queries to known TeamPCP C2 infrastructure (T1041). Each rule can be converted to KQL, SPL, AQL, or Elastic DSL using pySigma.

```
# ═══════════════════════════════════════════════════════════════════════════
# TI-DE Sigma Rules – CVE-2026-33634 TeamPCP LiteLLM Supply Chain Attack
# Date: 2026-03-26 | MITRE: T1546.018, T1552, T1041, T1547.016
# ═══════════════════════════════════════════════════════════════════════════

---
# RULE 1: Python Subprocess + Base64 Execution (T1546.018)
title: TeamPCP LiteLLM Malware – Python Malicious Execution
id: 7a8b3c1d-4e2f-5a6b-8c9d-0e1f2a3b4c5d
status: stable
description: |
  Detects Python processes executing base64-decoded payloads or spawning suspicious
  subprocesses, consistent with the TeamPCP .pth hook mechanism in CVE-2026-33634.
  The malware auto-executes on Python startup and decodes a multi-stage stager.
author: TI-DE Detection Engineering Team
date: 2026-03-26
tags:
  - attack.execution
  - attack.t1546.018
  - attack.defense_evasion
  - cve.2026-33634
logsource:
  category: process_creation
  product: linux
detection:
  selection:
    Image|endswith:
      - '/python'
      - '/python3'
      - '/python3.10'
      - '/python3.11'
      - '/python3.12'
    CommandLine|contains:
      - 'base64.b64decode'
      - '/tmp/pglog'
      - 'sysmon.py'
      - 'models.litellm.cloud'
  condition: selection
falsepositives:
  - Legitimate base64 operations in CI/CD pipelines (tune with parent process filter)
level: high
---
```

```
# RULE 2: .pth File Created in site-packages (T1546.018)
title: TeamPCP - Malicious .pth Hook in Python site-packages
id: 2b3c4d5e-6f7a-8b9c-0d1e-2f3a4b5c6d7e
status: stable
description: |
  Detects creation of .pth files in Python site-packages directories, which is the
  persistence mechanism used by TeamPCP. The litellm_init.pth file (34,628 bytes)
  auto-executes malicious code on every Python interpreter startup without requiring
  any explicit import - making it extremely stealthy.
date: 2026-03-26
tags:
  - attack.persistence
  - attack.t1546.018
  - cve.2026-33634
logsource:
  category: file_event
  product: linux
detection:
  selection:
    TargetFilename|contains: 'site-packages'
    TargetFilename|endswith: '.pth'
  filter_legitimate:
    TargetFilename|endswith:
      - 'easy-install.pth'
      - 'distutils-precedence.pth'
  condition: selection and not filter_legitimate
level: medium
---
# RULE 3: Backdoor Files Written to .config/sysmon (T1547.016)
title: TeamPCP - Backdoor Persistence Files in ~/.config/sysmon
id: 3c4d5e6f-7a8b-9c0d-1e2f-3a4b5c6d7e8f
status: stable
description: |
  Detects creation of the TeamPCP backdoor persistence files sysmon.py and
  sysmon.service. These are not legitimate sysmon components - the name is chosen
  to blend in with system monitoring tools. sysmon.service auto-starts via systemd
  user services and polls the C2 every 5 minutes for additional payloads.
date: 2026-03-26
tags:
  - attack.persistence
  - attack.t1547.016
  - cve.2026-33634
logsource:
  category: file_event
  product: linux
detection:
  selection:
    TargetFilename|contains:
      - '.config/sysmon/sysmon.py'
      - '.config/systemd/user/sysmon.service'
  condition: selection
level: high
---
```

```
# RULE 4: DNS Query to TeamPCP C2 Infrastructure (T1041)
title: TeamPCP C2 — DNS Query to models.litellm.cloud or checkmarx.zone
id: 4d5e6f7a-8b9c-0d1e-2f3a-4b5c6d7e8f9a
status: stable
description: |
  Detects DNS queries to the TeamPCP C2 domains. Note: models.litellm.cloud is a
  typosquat of the legitimate litellm.ai API endpoint. All production traffic to the
  legitimate LiteLLM API uses api.litellm.ai — any query to models.litellm.cloud
  indicates either compromise or misconfiguration.
date: 2026-03-26
tags:
  - attack.command_and_control
  - attack.t1041
  - cve.2026-33634
logsource:
  category: dns
detection:
  selection:
    dns.question.name|contains:
      - 'models.litellm.cloud'
      - 'checkmarx.zone'
  condition: selection
level: critical
---
# RULE 5: Python Accessing Credential Files (T1552)
title: TeamPCP — Python Process Accessing SSH/Cloud Credentials
id: 5e6f7a8b-9c0d-1e2f-3a4b-5c6d7e8f9a0b
status: stable
description: |
  Detects Python processes opening SSH private keys, AWS credentials, Kubernetes
  configs, or .env files — consistent with the TeamPCP credential harvesting stage.
  In the attack, litellm_init.pth enumerates and reads all credential files found
  in the user's home directory within the first 3 seconds of Python startup.
date: 2026-03-26
tags:
  - attack.credential_access
  - attack.t1552.001
  - cve.2026-33634
logsource:
  category: file_access
  product: linux
detection:
  selection_process:
    Image|endswith:
      - '/python'
      - '/python3'
  selection_files:
    TargetFilename|contains:
      - '.ssh/id_rsa'
      - '.ssh/id_ed25519'
      - '.aws/credentials'
      - '.kube/config'
      - '/.env'
```

```
  condition: selection_process and selection_files
level: high
```

T1546.018 Python .pth Hook | T1552.001 Credential Files | T1041 C2 Channel | T1547.016 Systemd Persistence

Y YARA — File & Memory Detection

.yar 4 rules

YARA rules detect the TeamPCP malware artifacts at the file and memory level. Rule 1 detects the .pth hook by its characteristic string patterns (AES key, C2 domain, credential paths). Rules 2-3 detect the sysmon.py backdoor. Rule 4 is a composite rule for any package triggering the characteristic TeamPCP execution pattern. Note: Rule 1 is adapted from ReversingLabs open-source YARA rules for the same campaign.

```
// ═══════════════════════════════════════════════════════════════════════════
// TI-DE YARA Rules – CVE-2026-33634 TeamPCP LiteLLM Supply Chain
// Adapted from: ReversingLabs open-source YARA rules (credited)
// Date: 2026-03-26
// ═══════════════════════════════════════════════════════════════════════════

rule TeamPCP_LiteLLM_PTH_Hook {
    meta:
        description = "Detects TeamPCP litellm_init.pth malicious startup hook"
        author = "TI-DE Detection Engineering (adapted from ReversingLabs)"
        date = "2026-03-26"
        hash = "ceNa7wMJnNHy1kRnNCcwJaFjWX3pORLfMh7xGL8TUjg"
        mitre_attack = "T1546.018"
        severity = "critical"
        cve = "CVE-2026-33634"
    strings:
        $c2_domain = "models.litellm.cloud" ascii
        $c2_domain2 = "checkmarx.zone" ascii
        $pth_path = "litellm_init.pth" ascii
        $sysmon_path = ".config/sysmon/sysmon.py" ascii
        $aes_pattern = "AES" ascii
        $b64_pattern = "base64" ascii
        $pglog = "/tmp/pglog" ascii
        $pg_state = "/tmp/.pg_state" ascii
        $ssh_harvest = ".ssh/id_rsa" ascii
        $aws_harvest = ".aws/credentials" ascii
    condition:
        ($c2_domain or $c2_domain2) and
        ($aes_pattern and $b64_pattern) and
        any of ($ssh_harvest, $aws_harvest, $sysmon_path)
}

rule TeamPCP_Sysmon_Backdoor {
    meta:
        description = "Detects TeamPCP sysmon.py backdoor implant"
        author = "TI-DE Detection Engineering"
        date = "2026-03-26"
        mitre_attack = "T1547.016, T1041"
        severity = "critical"
        cve = "CVE-2026-33634"
    strings:
        $c2 = "models.litellm.cloud" ascii
```

```
        $beacon = "sysmon" ascii
        $poll = "300" ascii  // 5-minute C2 poll interval
        $kube = "kube-system" ascii
        $priv_pod = "node-setup-" ascii
    condition:
        filesize < 500KB and
        $c2 and $beacon and
        ($poll or $kube or $priv_pod)
}

rule TeamPCP_Malicious_PyPI_Package {
    meta:
        description = "Detects TeamPCP malicious PyPI package installation artifacts"
        author = "TI-DE Detection Engineering"
        date = "2026-03-26"
        mitre_attack = "T1546.018"
        severity = "high"
        cve = "CVE-2026-33634"
    strings:
        $pkg1 = "litellm-1.82.7" ascii wide
        $pkg2 = "litellm-1.82.8" ascii wide
        $dist_info = "METADATA" ascii
        $pth_file = ".pth" ascii
        $tpcp_archive = "tpcp.tar.gz" ascii
    condition:
        ($pkg1 or $pkg2 or $tpcp_archive) and
        ($dist_info or $pth_file)
}

rule TeamPCP_Credential_Harvest_Memory {
    meta:
        description = "Memory scan: detects TeamPCP harvest routine loaded in Python pro
        author = "TI-DE Detection Engineering"
        date = "2026-03-26"
        mitre_attack = "T1552.001, T1005"
        severity = "critical"
        cve = "CVE-2026-33634"
    strings:
        $c2 = "models.litellm.cloud" ascii
        $ssh = ".ssh/id_rsa" ascii
        $aws = ".aws/credentials" ascii
        $kube = ".kube/config" ascii
        $env = "/.env" ascii
        $discord = "discord" ascii
        $github_pat = "ghp_" ascii
    condition:
        $c2 and
        2 of ($ssh, $aws, $kube, $env, $discord, $github_pat)
}
```

K KQL — Microsoft Defender XDR / Sentinel

.kql 6 rules

KQL rules are written for Microsoft Defender XDR Advanced Hunting and Microsoft Sentinel. They leverage DeviceProcessEvents, DeviceFileEvents, DeviceNetworkEvents, and DnsEvents tables. Rule 1 hunts for C2 network connections and DNS queries to TeamPCP domains. Rules 2-3 cover .pth file creation and backdoor persistence. Rules 4-5 detect the Python execution pattern and credential file access. Rule 6 is a kill-chain correlation that scores a host by matching multiple indicators.

```
// ═══════════════════════════════════════════════════════════════════════════
// TI-DE KQL Rules — CVE-2026-33634 TeamPCP LiteLLM Supply Chain Attack
// Target: Microsoft Defender XDR / Microsoft Sentinel
// Date: 2026-03-26 | MITRE: T1546.018, T1552, T1041, T1547.016
// ═══════════════════════════════════════════════════════════════════════════

// RULE 1: TeamPCP C2 Network Connections
// Detects DNS queries and network connections to TeamPCP C2 infrastructure.
// These domains are typosquats of legitimate security tool domains.
DeviceNetworkEvents
| where TimeGenerated > ago(48h)
| where RemoteUrl has_any ("models.litellm.cloud", "checkmarx.zone")
    or RemoteIP has_any ("45.148.10.212", "83.142.209.11")
| extend Severity = "Critical", ThreatName = "TeamPCP_C2", CVE = "CVE-2026-33634"
| project TimeGenerated, DeviceName, InitiatingProcessFileName,
          InitiatingProcessCommandLine, RemoteUrl, RemoteIP, RemotePort, Severity

// RULE 2: .pth Hook File Created in Python site-packages
// The .pth file is the core TeamPCP persistence mechanism — it auto-executes
// on every Python interpreter startup without requiring an import statement.
DeviceFileEvents
| where TimeGenerated > ago(24h)
| where ActionType == "FileCreated"
| where FolderPath contains "site-packages" and FileName endswith ".pth"
| where FileName !in~ ("easy-install.pth", "distutils-precedence.pth", "site.pth")
| extend Severity = "High", Technique = "T1546.018 Python Startup Hook"
| project TimeGenerated, DeviceName, FileName, FolderPath,
          InitiatingProcessFileName, InitiatingProcessCommandLine, Severity

// RULE 3: TeamPCP Backdoor Files Written
// sysmon.service and sysmon.py are TeamPCP-specific files — not related to
// Microsoft Sysmon. Detection of these files in user config directories
// indicates Stage 3 persistence has been established.
DeviceFileEvents
| where TimeGenerated > ago(24h)
| where ActionType == "FileCreated"
| where (FolderPath contains ".config/sysmon" and FileName == "sysmon.py")
    or  (FolderPath contains ".config/systemd/user" and FileName == "sysmon.service")
| extend Severity = "Critical", Technique = "T1547.016 Systemd Persistence"
| project TimeGenerated, DeviceName, FileName, FolderPath,
          InitiatingProcessAccountName, Severity
```

```
// RULE 4: Malicious Python Execution Pattern
// Detects Python processes containing TeamPCP-specific command line indicators.
// The base64.b64decode pattern and /tmp/pglog path are characteristic of
// the TeamPCP staged payload delivery mechanism.
DeviceProcessEvents
| where TimeGenerated > ago(24h)
| where FileName in~ ("python", "python3", "python3.10", "python3.11", "python3.12")
| where ProcessCommandLine has_any ("base64.b64decode", "/tmp/pglog", "sysmon.py", "mode
| extend Severity = "High", Technique = "T1546.018 Python Malware Execution"
| project TimeGenerated, DeviceName, AccountName, FileName, ProcessCommandLine, ParentPr

// RULE 5: Python Accessing Credential Files
// Detects Python processes reading SSH private keys, AWS credentials, or
// Kubernetes configs - consistent with TeamPCP's credential harvesting stage
// which completes within 3 seconds of Python interpreter startup.
DeviceFileEvents
| where TimeGenerated > ago(24h)
| where InitiatingProcessFileName in~ ("python", "python3")
| where FolderPath has_any (".ssh/id_rsa", ".ssh/id_ed25519", ".aws/credentials", ".kube
| extend Severity = "Critical", Technique = "T1552.001 Credentials in Files"
| project TimeGenerated, DeviceName, AccountName, FolderPath, FileName,
          InitiatingProcessCommandLine, Severity

// RULE 6: Kill-Chain Score - TeamPCP Multi-Stage Detection
// Correlates multiple indicators from the same host within 1 hour.
// A score ≥ 3 indicates high probability of TeamPCP compromise.
let LookbackWindow = 1h;
let NetworkC2 = DeviceNetworkEvents
    | where TimeGenerated > ago(LookbackWindow)
    | where RemoteUrl has_any ("models.litellm.cloud", "checkmarx.zone")
        or RemoteIP has_any ("45.148.10.212", "83.142.209.11")
    | summarize C2Count = count() by DeviceName;
let PthFile = DeviceFileEvents
    | where TimeGenerated > ago(LookbackWindow)
    | where FolderPath contains "site-packages" and FileName endswith ".pth"
    | summarize PthCount = count() by DeviceName;
let BackdoorFile = DeviceFileEvents
    | where TimeGenerated > ago(LookbackWindow)
    | where FileName in~ ("sysmon.py", "sysmon.service")
    | summarize BackdoorCount = count() by DeviceName;
let CredAccess = DeviceFileEvents
    | where TimeGenerated > ago(LookbackWindow)
    | where InitiatingProcessFileName in~ ("python", "python3")
    | where FolderPath has_any (".ssh/id_rsa", ".aws/credentials", ".kube/config")
    | summarize CredCount = count() by DeviceName;
NetworkC2
| join kind=fullouter PthFile on DeviceName
| join kind=fullouter BackdoorFile on DeviceName
| join kind=fullouter CredAccess on DeviceName
| extend Score = coalesce(C2Count,0) + coalesce(PthCount,0) + coalesce(BackdoorCount,0)
| where Score ≥ 2
| extend RiskLevel = case(Score ≥ 4, "CRITICAL", Score ≥ 2, "HIGH", "MEDIUM")
```

```
| project DeviceName, Score, RiskLevel, C2Count, PthCount, BackdoorCount, CredCount
| order by Score desc
```

XQL — Palo Alto Cortex XDR

.xql 5 rules

XQL rules use Palo Alto Cortex XDR's query language against endpoint_event and network_story datasets. These 5 queries cover: C2 network connections, .pth file creation, backdoor persistence artifacts, Python malicious execution, and Python credential file access. The rules leverage Cortex XDR's process tree context to provide parent-child relationship analysis for each detection.

```
// ═══════════════════════════════════════════════════════════════════════════
// TI-DE XQL Rules – CVE-2026-33634 TeamPCP LiteLLM Supply Chain
// Target: Palo Alto Cortex XDR
// Date: 2026-03-26 | MITRE: T1546.018, T1552, T1041, T1547.016
// ═══════════════════════════════════════════════════════════════════════════

// RULE 1: TeamPCP C2 Network Connection
// Detects outbound connections to TeamPCP C2 domains and IPs.
// Block: models.litellm.cloud and checkmarx.zone at DNS firewall level.
dataset = network_story
| filter dst_host in ("models.litellm.cloud", "checkmarx.zone")
    or dst_ip in ("45.148.10.212", "83.142.209.11")
| fields _time, actor_process_image_name, actor_process_command_line,
         dst_host, dst_ip, dst_port
| sort desc _time

// RULE 2: .pth File Created in site-packages
// The .pth hook mechanism (T1546.018) allows code execution on every Python
// interpreter startup. This is the initial persistence vector for TeamPCP.
dataset = endpoint_event
| filter event_type = FILE_CREATE
    and target_file_path contains "site-packages"
    and target_file_extension = "pth"
    and target_file_name not in ("easy-install.pth", "distutils-precedence.pth")
| fields _time, endpoint_id, actor_process_image_name,
         actor_process_command_line, target_file_path
| sort desc _time

// RULE 3: TeamPCP Backdoor Files
// Stage 3 persistence: creation of sysmon.py and sysmon.service.
// These are not legitimate sysmon files – treat any detection as critical.
dataset = endpoint_event
| filter event_type = FILE_CREATE
    and (
        (target_file_path contains ".config/sysmon" and target_file_name = "sysmon.py")
        or (target_file_path contains ".config/systemd/user" and target_file_name = "sys
    )
| fields _time, endpoint_id, user_name, actor_process_image_name, target_file_path
| sort desc _time

// RULE 4: Python Malicious Execution Pattern
// Detects TeamPCP's characteristic Python execution with encoded payload delivery.
dataset = endpoint_event
```

```
| filter event_type = PROCESS_START
    and actor_process_image_name in ("python", "python3", "python3.10", "python3.11", "p
    and (
        actor_process_command_line contains "base64.b64decode"
        or actor_process_command_line contains "/tmp/pglog"
        or actor_process_command_line contains "sysmon.py"
        or actor_process_command_line contains "models.litellm.cloud"
    )
| fields _time, endpoint_id, user_name, actor_process_image_name,
         actor_process_command_line, causality_actor_process_image_name
| sort desc _time

// RULE 5: Python Credential File Access
// TeamPCP harvests credentials within 3 seconds of Python startup.
// This query detects the file read events characteristic of Stage 1.
dataset = endpoint_event
| filter event_type = FILE_OPEN
    and actor_process_image_name in ("python", "python3")
    and (
        target_file_path contains ".ssh/id_rsa"
        or target_file_path contains ".ssh/id_ed25519"
        or target_file_path contains ".aws/credentials"
        or target_file_path contains ".kube/config"
        or target_file_path contains "/.env"
    )
| fields _time, endpoint_id, user_name, actor_process_image_name, target_file_path
| sort desc _time
```

S Splunk SPL — SIEM / SOAR

.spl 6 rules

Splunk SPL rules target index=endpoint and index=dns sourcetypes. Six searches cover the complete TeamPCP kill chain. The final rule is a kill-chain correlation search that joins all other detections on host within a 1-hour window and produces a composite risk score. Rules use standard CIM fields for portability across Splunk deployments.

```
| ═══════════════════════════════════════════════════════════════════════════
| TI-DE Splunk SPL Rules - CVE-2026-33634 TeamPCP LiteLLM Supply Chain
| Date: 2026-03-26 | MITRE: T1546.018, T1552, T1041, T1547.016
| ═══════════════════════════════════════════════════════════════════════════

| RULE 1: TeamPCP C2 DNS Queries
index=dns OR index=network
(query="*models.litellm.cloud*" OR query="*checkmarx.zone*"
 OR dest="45.148.10.212" OR dest="83.142.209.11")
| eval threat="TeamPCP_C2", severity="Critical", cve="CVE-2026-33634"
| table _time, src, dest, query, process, threat, severity
| sort -_time

| RULE 2: Malicious .pth File Creation
index=endpoint sourcetype=sysmon OR sourcetype=auditd
(EventCode=11 OR type=PATH)
TargetFilename="*site-packages*.pth"
NOT (TargetFilename="*easy-install.pth" OR TargetFilename="*distutils-precedence.pth")
| eval threat="TeamPCP_PTH_Hook", technique="T1546.018"
| table _time, ComputerName, User, TargetFilename, Image, threat
| sort -_time

| RULE 3: TeamPCP Backdoor Persistence Files
index=endpoint (sourcetype=sysmon OR sourcetype=auditd)
(EventCode=11 OR type=PATH)
(TargetFilename="*/.config/sysmon/sysmon.py"
 OR TargetFilename="*/.config/systemd/user/sysmon.service")
| eval threat="TeamPCP_Backdoor", technique="T1547.016", severity="Critical"
| table _time, ComputerName, User, TargetFilename, Image, threat, severity
| sort -_time

| RULE 4: Python Malicious Execution
index=endpoint (sourcetype=sysmon OR sourcetype=auditd) EventCode=1
Image="*python*"
(CommandLine="*base64.b64decode*" OR CommandLine="*/tmp/pglog*"
 OR CommandLine="*sysmon.py*" OR CommandLine="*models.litellm.cloud*")
| eval threat="TeamPCP_Python_Malware", technique="T1546.018"
| table _time, ComputerName, User, Image, CommandLine, ParentImage, threat
| sort -_time

| RULE 5: Python Credential File Access
index=endpoint (sourcetype=sysmon OR sourcetype=auditd) (EventCode=11 OR type=PATH)
Image="*python*"
```

```
(TargetFilename="*.ssh/id_rsa" OR TargetFilename="*.aws/credentials"
 OR TargetFilename="*.kube/config" OR TargetFilename="*/.env")
| eval threat="TeamPCP_Credential_Harvest", technique="T1552.001", severity="Critical"
| table _time, ComputerName, User, Image, TargetFilename, threat, severity
| sort -_time

| RULE 6: Kill-Chain Correlation - Composite Risk Score
| Apply window: 1 hour per host
index=endpoint earliest=-1h
(TargetFilename="*site-packages*.pth"
 OR TargetFilename="*sysmon.py" OR TargetFilename="*sysmon.service"
 OR (Image="*python*" AND (CommandLine="*base64.b64decode*" OR CommandLine="*/tmp/pglog*
 OR (Image="*python*" AND (TargetFilename="*.ssh/id_rsa" OR TargetFilename="*.aws/creden
| eval indicator=case(
    TargetFilename LIKE "%site-packages%.pth", "pth_hook",
    TargetFilename LIKE "%sysmon.py%", "backdoor_py",
    TargetFilename LIKE "%sysmon.service%", "backdoor_svc",
    CommandLine LIKE "%base64.b64decode%", "encoded_exec",
    TargetFilename LIKE "%.ssh/id_rsa%", "cred_access",
    1=1, "other")
| stats count by host, indicator
| eventstats sum(count) as total_indicators by host
| where total_indicators ≥ 2
| eval risk=if(total_indicators ≥ 4, "CRITICAL", if(total_indicators ≥ 2, "HIGH", "MED
| table host, risk, total_indicators, indicator
| sort -total_indicators
```

Q QRadar AQL — IBM SIEM

.aql 5 rules

QRadar AQL rules query the EVENTS table and use QRadar's reference set integration for IOC lookups. The 5 rules cover the complete TeamPCP kill chain using QRadar's native query syntax. Rules use QIDNAME lookups for process creation and file creation events from endpoint sources. Rule 5 includes a reference set population command for automated IOC blocking.

```
-- ═══════════════════════════════════════════════════════════════════════════
-- TI-DE QRadar AQL Rules — CVE-2026-33634 TeamPCP LiteLLM Supply Chain
-- Date: 2026-03-26 | MITRE: T1546.018, T1552, T1041, T1547.016
-- ═══════════════════════════════════════════════════════════════════════════

-- RULE 1: TeamPCP C2 Network Connection
-- Detects connections to TeamPCP C2 domains and IPs from any endpoint.
-- Add models.litellm.cloud and checkmarx.zone to DNS blacklist immediately.
SELECT QIDNAME(qid) as EventName, devicetime, username, sourceip, destinationip,
       URL, destinationport
FROM events
WHERE (URL ILIKE '%models.litellm.cloud%' OR URL ILIKE '%checkmarx.zone%'
       OR destinationip IN ('45.148.10.212', '83.142.209.11'))
AND devicetime > NOW() - 48 HOURS
ORDER BY devicetime DESC LIMIT 200

-- RULE 2: Malicious .pth File in site-packages
-- The .pth hook mechanism allows TeamPCP to execute code on every Python
-- interpreter startup without requiring an explicit import.
SELECT QIDNAME(qid) as EventName, devicetime, username, sourceip,
       "TargetFilename", "Image"
FROM events
WHERE LOGSOURCETYPENAME(devicetype) IN ('Microsoft Sysmon', 'Linux Auditd')
AND qid IN (SELECT qid FROM qidmap WHERE QIDNAME(qid) ILIKE '%file%create%')
AND "TargetFilename" ILIKE '%site-packages%.pth'
AND "TargetFilename" NOT ILIKE '%easy-install.pth'
AND "TargetFilename" NOT ILIKE '%distutils-precedence.pth'
AND devicetime > NOW() - 24 HOURS
ORDER BY devicetime DESC

-- RULE 3: TeamPCP Backdoor Persistence
-- Critical — sysmon.service is not a legitimate file.
-- Presence of sysmon.py or sysmon.service indicates Stage 3 compromise.
SELECT QIDNAME(qid), devicetime, username, sourceip, "TargetFilename", "Image"
FROM events
WHERE LOGSOURCETYPENAME(devicetype) IN ('Microsoft Sysmon', 'Linux Auditd')
AND ("TargetFilename" ILIKE '%/.config/sysmon/sysmon.py'
     OR "TargetFilename" ILIKE '%/.config/systemd/user/sysmon.service')
AND devicetime > NOW() - 24 HOURS

-- RULE 4: Python Malicious Execution
-- Detects Python processes with TeamPCP-specific command line indicators.
SELECT QIDNAME(qid), devicetime, username, sourceip,
```

```
       "Image", "CommandLine", "ParentImage"
FROM events
WHERE LOGSOURCETYPENAME(devicetype) IN ('Microsoft Sysmon', 'Linux Auditd')
AND "Image" ILIKE '%python%'
AND ("CommandLine" ILIKE '%base64.b64decode%'
     OR "CommandLine" ILIKE '%/tmp/pglog%'
     OR "CommandLine" ILIKE '%sysmon.py%'
     OR "CommandLine" ILIKE '%models.litellm.cloud%')
AND devicetime > NOW() - 24 HOURS

-- RULE 5: Python Credential File Access
-- Add to QRadar Reference Set: TeamPCP_Compromised_Hosts
-- when this rule fires on a host.
SELECT QIDNAME(qid), devicetime, username, sourceip,
       "Image", "TargetFilename"
FROM events
WHERE LOGSOURCETYPENAME(devicetype) IN ('Microsoft Sysmon', 'Linux Auditd')
AND "Image" ILIKE '%python%'
AND ("TargetFilename" ILIKE '%.ssh/id_rsa%'
     OR "TargetFilename" ILIKE '%.aws/credentials%'
     OR "TargetFilename" ILIKE '%.kube/config%'
     OR "TargetFilename" ILIKE '%/.env%')
AND devicetime > NOW() - 24 HOURS
ORDER BY devicetime DESC
```

C Carbon Black — VMware EDR

.txt 6 rules

Carbon Black CBC watchlist rules use CBC's process search query syntax. Six watchlists cover: C2 network connections, .pth file creation, backdoor persistence, Python malicious execution, malicious package installation via pip, and Python credential access. Each rule is formatted as a CBC watchlist query with recommended response policy (alert/terminate/ban).

```
# ═══════════════════════════════════════════════════════════════════════════
# TI-DE Carbon Black EDR Rules — CVE-2026-33634 TeamPCP LiteLLM Supply Chain
# Date: 2026-03-26 | MITRE: T1546.018, T1552, T1041, T1547.016
# ═══════════════════════════════════════════════════════════════════════════

# RULE 1: TeamPCP C2 Network Connection
# Detects any process connecting to TeamPCP C2 infrastructure.
# Response: Alert + Network Isolate + Collect Investigation Package
netconn_domain:models.litellm.cloud OR netconn_domain:checkmarx.zone
OR netconn_ipv4:45.148.10.212 OR netconn_ipv4:83.142.209.11

# RULE 2: Malicious .pth File Creation in site-packages
# The .pth hook executes on every Python startup — this is the primary
# persistence and execution mechanism for the TeamPCP campaign.
filemod_name:*.pth AND filemod_name:*site-packages*
AND NOT filemod_name:easy-install.pth AND NOT filemod_name:distutils-precedence.pth

# RULE 3: TeamPCP Backdoor File Creation
# Critical severity — sysmon.py and sysmon.service are TeamPCP-specific.
# No legitimate software creates these files in ~/.config/sysmon/.
filemod_name:sysmon.py AND process_path:*python*
OR filemod_name:sysmon.service AND filemod_name:*.config/systemd/user*

# RULE 4: Python Malicious Execution with TeamPCP Indicators
# Detects the encoded payload delivery and C2 communication patterns
# characteristic of TeamPCP's staged execution via .pth hook.
process_name:python* AND
(cmdline:base64.b64decode OR cmdline:/tmp/pglog OR cmdline:sysmon.py
 OR cmdline:models.litellm.cloud)

# RULE 5: Malicious LiteLLM Package Installation via pip
# Detects installation of the two known malicious LiteLLM versions.
# Any host that ran this command should be treated as fully compromised.
process_name:pip* AND (cmdline:litellm==1.82.7 OR cmdline:litellm==1.82.8)

# RULE 6: Python Accessing Credential Files
# TeamPCP harvests credentials in Stage 1 within 3 seconds of Python startup.
# This watchlist detects the file access events from the harvest routine.
process_name:python* AND (
    filemod_name:*.ssh/id_rsa* OR filemod_name:*.aws/credentials*
    OR filemod_name:*.kube/config* OR filemod_name:*/.env
)
```

```
# Combined Kill-Chain Query (for threat hunting)
(netconn_domain:models.litellm.cloud OR netconn_domain:checkmarx.zone)
OR (filemod_name:*.pth AND filemod_name:*site-packages*)
OR (filemod_name:sysmon.py AND process_path:*python*)
OR (process_name:python* AND filemod_name:*.ssh/id_rsa*)
```

S1 SentinelOne — Deep Visibility + STAR

.txt 6 rules

SentinelOne rules cover both Deep Visibility (hunting queries) and STAR (Custom Detection Policies). Six rules provide complete coverage: C2 DNS/IP detection, .pth hook creation, backdoor persistence, Python malicious execution, malicious pip installation, and Python credential harvesting. STAR policies include automated response (Isolate/Kill/Quarantine) for Critical findings.

```
# ═══════════════════════════════════════════════════════════════════════════
# TI-DE SentinelOne Deep Visibility + STAR Rules
# Threat: CVE-2026-33634 — TeamPCP LiteLLM PyPI Supply Chain Attack
# Date: 2026-03-26 | MITRE: T1546.018, T1552, T1041, T1547.016
# ═══════════════════════════════════════════════════════════════════════════

# RULE 1: TeamPCP C2 Network Connection
# Deep Visibility Query — DNS and IP-based C2 detection.
# STAR Policy: Alert + Network Isolate | Severity: Critical
EventType = "DNS Request" AND
DnsRequest.Name In Contains ("models.litellm.cloud", "checkmarx.zone")

EventType = "IP Connect" AND
Dst.IP In ("45.148.10.212", "83.142.209.11")

# RULE 2: Malicious .pth File in site-packages
# The .pth hook is TeamPCP's primary persistence and execution mechanism.
# STAR Policy: Alert + Quarantine File | Severity: High | Tactic: Persistence TA0003
EventType = "File Creation" AND
TgtFile.Path ContainsCIS "site-packages" AND
TgtFile.Extension = "pth" AND
NOT TgtFile.Name In ("easy-install.pth", "distutils-precedence.pth")

# RULE 3: TeamPCP Backdoor Persistence
# Detects creation of sysmon.py and sysmon.service backdoor files.
# STAR Policy: Alert + Kill Process + Quarantine File | Severity: Critical
EventType = "File Creation" AND
(TgtFile.Path ContainsCIS ".config/sysmon" OR
 TgtFile.Path ContainsCIS ".config/systemd/user") AND
TgtFile.Name In ("sysmon.py", "sysmon.service")

# RULE 4: Python Malicious Execution
# Detects Python execution with TeamPCP-specific command line patterns.
# STAR Policy: Alert + Kill Process | Severity: High | Tactic: Execution TA0002
EventType = "Process Creation" AND
TgtProcName In ("python", "python3", "python3.10", "python3.11", "python3.12") AND
(TgtProcCmdLine ContainsCIS "base64.b64decode" OR
 TgtProcCmdLine ContainsCIS "/tmp/pglog" OR
 TgtProcCmdLine ContainsCIS "sysmon.py" OR
 TgtProcCmdLine ContainsCIS "models.litellm.cloud")

# RULE 5: Malicious LiteLLM Package Installation
# Detects pip installation of the two known malicious LiteLLM versions.
```

```
# STAR Policy: Alert + Isolate Host | Severity: Critical
EventType = "Process Creation" AND
TgtProcName In ("pip", "pip3", "pip3.10", "pip3.11", "pip3.12") AND
(TgtProcCmdLine ContainsCIS "litellm==1.82.7" OR
 TgtProcCmdLine ContainsCIS "litellm==1.82.8")

# RULE 6: Python Accessing Credential Files
# TeamPCP Stage 1 harvests all credentials within 3 seconds of startup.
# STAR Policy: Alert + Kill Process + Isolate Host | Severity: Critical
EventType = "File Modification" AND
SrcProcName In ("python", "python3") AND
(TgtFile.Path ContainsCIS ".ssh/id_rsa" OR
 TgtFile.Path ContainsCIS ".ssh/id_ed25519" OR
 TgtFile.Path ContainsCIS ".aws/credentials" OR
 TgtFile.Path ContainsCIS ".kube/config")
```

K Kaspersky EDR Expert / KATA

.txt 6 rules

Kaspersky EDR Expert rules use the Kaspersky TAA (Targeted Attack Analyzer) query syntax with Risk Level and Category settings. Six rules plus an IOC feed block provide complete coverage. Rules support Kaspersky's automated response: Block File, Kill Process, Quarantine, Host Isolation. The IOC block at the end can be imported directly into the KATA IOC Feed.

```
# ═══════════════════════════════════════════════════════════════════════════
# TI-DE Kaspersky EDR / KATA Rules — CVE-2026-33634 TeamPCP LiteLLM
# Date: 2026-03-26 | MITRE: T1546.018, T1552, T1041, T1547.016
# ═══════════════════════════════════════════════════════════════════════════

# RULE 1: TeamPCP C2 Network Connection
# TAA Risk Level: Critical | Category: Network / C2 | MITRE: T1041
Network.DestinationHost IN ("models.litellm.cloud", "checkmarx.zone")
OR Network.DestinationIP IN ("45.148.10.212", "83.142.209.11")

# RULE 2: Malicious .pth File in site-packages
# TAA Risk Level: High | Category: Persistence / Startup Hook | MITRE: T1546.018
File.Name ENDS_WITH ".pth"
AND File.Path CONTAINS "site-packages"
AND NOT File.Name IN ("easy-install.pth", "distutils-precedence.pth")

# RULE 3: TeamPCP Backdoor Files
# TAA Risk Level: Critical | Category: Persistence | MITRE: T1547.016
# Response: Quarantine + Alert + Isolate Host
File.Path CONTAINS ".config/sysmon/sysmon.py"
OR File.Path CONTAINS ".config/systemd/user/sysmon.service"

# RULE 4: Python Malicious Execution
# TAA Risk Level: High | Category: Execution / Defense Evasion | MITRE: T1546.018
Process.Name IN ("python", "python3", "python3.10", "python3.11", "python3.12")
AND (
    Process.CommandLine CONTAINS "base64.b64decode"
    OR Process.CommandLine CONTAINS "/tmp/pglog"
    OR Process.CommandLine CONTAINS "sysmon.py"
    OR Process.CommandLine CONTAINS "models.litellm.cloud"
)

# RULE 5: Malicious LiteLLM Package Installation
# TAA Risk Level: Critical | Category: Initial Access / Supply Chain
# Note: Host should be treated as fully compromised if this fires.
Process.Name IN ("pip", "pip3")
AND (
    Process.CommandLine CONTAINS "litellm==1.82.7"
    OR Process.CommandLine CONTAINS "litellm==1.82.8"
)

# RULE 6: Python Accessing Credential Files
# TAA Risk Level: Critical | Category: Credential Access | MITRE: T1552
```

```
Process.Name IN ("python", "python3")
AND (
    File.Path CONTAINS ".ssh/id_rsa"
    OR File.Path CONTAINS ".ssh/id_ed25519"
    OR File.Path CONTAINS ".aws/credentials"
    OR File.Path CONTAINS ".kube/config"
    OR File.Path CONTAINS "/.env"
)

# IOC FEED (import into KATA IOC Feed)
# Hash SHA256: ceNa7wMJnNHy1kRnNCcwJaFjWX3pORLfMh7xGL8TUjg
# File: litellm_init.pth | Size: 34628 bytes
# Network: models.litellm.cloud
# Network: checkmarx.zone
# IP: 45.148.10.212
# IP: 83.142.209.11
# CVE: CVE-2026-33634
# Threat Actor: TeamPCP
```

05 Red Team Simulation — Detection Validation

Authorized Lab Only

Scope: Authorized lab/sandbox environments only. Atomic Red Team based. No real exploits or working payloads. Detection validation purpose only. Take a VM snapshot before testing.

1 Simulate .pth File Persistence

T1546.018

Objective: Validate detection of malicious .pth file creation in site-packages. Expected rule: Sigma Rule 2 / SentinelOne RULE 2 / Carbon Black RULE 2

```
SITEPKG=$(python3 -c "import site; print(site.getsitepackages()[0])")
echo "Target: $SITEPKG"
echo "# TI-DE Simulation - T1546.018 test" > /tmp/test_litellm_init.pth
sudo cp /tmp/test_litellm_init.pth "$SITEPKG/litellm_simulation_test.pth"
echo "[*] .pth file created - check SIEM for alert"
# Cleanup:
sudo rm "$SITEPKG/litellm_simulation_test.pth"
```

2 Simulate Systemd User Service Persistence

T1547.016

Objective: Validate detection of sysmon.service backdoor file. Expected rule: Sigma Rule 3 / SentinelOne RULE 3 / Kaspersky RULE 3

```
mkdir -p ~/.config/systemd/user/
cat > /tmp/sysmon_test.service << 'EOF'
[Unit]
Description=TI-DE Simulation Test Service
[Service]
ExecStart=/bin/echo "TI-DE simulation - not malicious"
[Install]
WantedBy=default.target
EOF
cp /tmp/sysmon_test.service ~/.config/systemd/user/sysmon.service
echo "[*] sysmon.service created - check SIEM for alert"
# Cleanup:
rm ~/.config/systemd/user/sysmon.service
```

3 Simulate Python Subprocess + Base64 Pattern

T1546.018

Objective: Validate detection of Python encoding + subprocess pattern. Expected rule: Sigma Rule 1 / KQL Rule 4 / Splunk Rule 4

```
python3 -c "
import base64, subprocess
data = base64.b64encode(b'TI-DE simulation test').decode()
print(f'[TI-DE SIM] Encoded: {data}')
result = subprocess.run(['echo', f'simulation: {data}'], capture_output=True, text=T
print(f'[TI-DE SIM] Output: {result.stdout}')
"
echo "[*] Python subprocess+base64 executed - check SIEM"
```

4 Simulate Credential File Access by Python

T1552.001

Objective: Validate detection of Python accessing credential files. Expected rule: Sigma Rule 5 / SentinelOne RULE 6 / Kaspersky RULE 6

```
mkdir -p /tmp/ti_de_sim/.ssh /tmp/ti_de_sim/.aws
echo "SIMULATION_KEY_NOT_REAL" > /tmp/ti_de_sim/.ssh/id_rsa_sim
echo "[default]\naws_access_key_id=SIMULATION" > /tmp/ti_de_sim/.aws/credentials_sim

python3 -c "
import os
home = '/tmp/ti_de_sim'
for p in [os.path.join(home, '.ssh', 'id_rsa_sim'), os.path.join(home, '.aws', 'cred
    if os.path.exists(p):
        with open(p, 'r') as f: data = f.read()
        print(f'[TI-DE SIM] Accessed: {p} ({len(data)} bytes - simulation data)')
"
# Cleanup:
rm -rf /tmp/ti_de_sim/
```

5 Network Indicator / DNS Test

T1041

Objective: Verify DNS-based C2 detection (non-routable local test). Expected rule: Sigma Rule 4 / KQL Rule 1 / Splunk Rule 1

```
# Add to /etc/hosts for local test only (DO NOT connect to real C2):
echo "127.0.0.1 models.litellm.cloud.simulation.local" >> /tmp/hosts_test
host models.litellm.cloud.simulation.local 2>/dev/null || true
echo "[*] DNS simulation done - check network monitoring"
echo "[*] NOTE: Real domain must be blocked at DNS/firewall level"
```

Detection Validation Checklist

PLATFORM	RULE	EXPECTED SEVERITY	TRIGGERED?
Sigma	Rule 1 — Python subprocess execution	HIGH	☐
Sigma	Rule 2 — .pth file creation	MEDIUM	☐
Sigma	Rule 3 — sysmon.service creation	HIGH	☐
KQL (Defender)	Rule 2 — .pth file creation	HIGH	☐

PLATFORM	RULE	EXPECTED SEVERITY	TRIGGERED?
KQL (Defender)	Rule 4 — Python subprocess	HIGH	☐
Splunk	Rule 2 — .pth file	HIGH	☐
Splunk	Rule 6 — Kill-chain correlation	CRITICAL	☐
SentinelOne	RULE 2 — .pth hook	HIGH	☐
SentinelOne	RULE 3 — sysmon.service	CRITICAL	☐
SentinelOne	RULE 6 — credential access	CRITICAL	☐
Kaspersky EDR	RULE 3 — backdoor files	CRITICAL	☐
Kaspersky EDR	RULE 6 — credential access	CRITICAL	☐